

Alexandria University
Faculty of Engineering
Computer and Systems Engineering Department

Assignment #4

Environmental Sound Classification

Using the UrbanSound8K Dataset with GRU-based Deep Learning Models

Name	Student ID
Omar Khaled Hussien	22010962
Yaseen Asaad Ahmed	22011349
Ahmed Mohamed Mansour	22010576

Contents

1	Introduction and Problem Statement	2
2	Dataset Description and Preprocessing	3
2.1	UrbanSound8K Dataset	3
2.2	Fold Splits	3
2.3	Audio Loading and Preprocessing	3
3	Data Exploration and Visualisation	4
3.1	Fold and Class Distribution	4
3.2	Waveform Analysis	5
3.3	Log-Mel Spectrograms	6
3.4	Gammatone Filterbank Spectrograms and GFCCs	6
3.5	Audio Listening	6
4	Feature Engineering	7
4.1	Feature Space Design Rationale	7
4.2	GFCCs — Gammatone Filterbank Cepstral Coefficients	8
4.3	Multi-scale Mel Spectrograms	8
4.4	Global Normalisation	9
4.5	Feature Caching	9
5	Data Augmentation	9
6	Model Architectures	10
6.1	GRU Classifier	11
6.2	Deep CNN GRU Classifier	11
6.3	Training Configuration	12
6.3.1	Label-Smoothed Cross-Entropy	13
6.3.2	Hyperparameter Grid	13
7	Results and Analysis	13
7.1	Learning Curves	13
7.2	Hyperparameter Comparison	15
7.3	Validation Confusion Matrices	15
7.4	Test Set Results	16

1 Introduction and Problem Statement

Environmental sound recognition is a fundamental problem in audio intelligence, with applications spanning urban noise monitoring, surveillance systems, autonomous navigation, and assistive technologies for the hearing-impaired. Unlike speech or music, environmental sounds are highly diverse, non-stationary, and often overlapping, making classification challenging even for humans.

This report documents our approach to classifying urban environmental sounds using the **UrbanSound8K** dataset. The core requirement is a classifier built on **Gated Recurrent Units (GRUs)**, leveraging their ability to model sequential dependencies in time-series data such as audio.

Assignment Goals

- Understand and explore the UrbanSound8K dataset.
- Engineer a rich acoustic feature space (Mel spectrograms, MFCCs, GFCCs, and more).
- Implement and train GRU-based classifiers with hyperparameter search.
- Implement a bonus CNN-GRU architecture and compare it against pure-GRU models.
- Evaluate all models with accuracy, macro F1-score, and confusion matrices.

Table 1: Summary of improvements over the GRU baseline.

Area	Baseline	This Work
Feature dimensionality	123 dims	565 dims (GFCCs, multi-scale Mel, $\Delta/\Delta\Delta$ MFCCs, ZCR, Rolloff)
Normalisation	Per-sample z-score	Global mean/std from training set (Welford)
Attention	Single-head	Multi-head (4 heads) + residual projection
CNN-GRU	Simple 2-layer CNN	Deep residual CNN (3 ResConv blocks, SE attention)
Loss	Cross-entropy	Label-smoothed CE ($\varepsilon = 0.1$)
Augmentation	4 basic ops	+ Mixup ($\alpha = 0.4$) + stronger SpecAugment
LR schedule	Cosine	Cosine + linear warmup + ReduceLROnPlateau fallback
Training budget	50 epochs max	100 epochs max (patience = 15)

2 Dataset Description and Preprocessing

2.1 UrbanSound8K Dataset

The UrbanSound8K dataset [?] contains **8,732 labelled audio clips** of urban sounds, each at most 4 seconds long, drawn from 10 categories. The clips are pre-organised into 10 folds that preserve the recording-source stratification (i.e. clips from the same source file are always in the same fold), enabling reproducible cross-validation.

Table 2: UrbanSound8K dataset overview.

Property	Value
Total clips	8,732
Number of classes	10
Maximum clip length	4 seconds
Native sample rate	22,050 Hz
Pre-defined folds	10

The 10 sound classes are: *air_conditioner*, *car_horn*, *children_playing*, *dog_bark*, *drilling*, *engine_idling*, *gun_shot*, *jackhammer*, *siren*, and *street_music*.

2.2 Fold Splits

Following the assignment specification, folds are assigned as follows:

Table 3: Train/validation/test fold assignments.

Split	Folds	Approx. clips	Approx. %
Training	1 – 6	$\approx 5,435$	62%
Validation	7 – 8	$\approx 1,644$	19%
Test	9 – 10	$\approx 1,653$	19%

Note

The test split is held out entirely during training and hyperparameter selection. Only the best model — determined by validation accuracy — is evaluated on the test folds.

2.3 Audio Loading and Preprocessing

Every audio file is loaded, resampled to 22,050 Hz (mono), and padded or trimmed to exactly 4 seconds (88,200 samples). Padding uses zero-padding at the end; trimming removes trailing samples.

```

1 def load_audio(filepath, sr=22050, duration=4.0):
2     """Load, resample, and pad/trim to exactly 'duration'
3         seconds."""
4     y, _ = librosa.load(str(filepath), sr=sr, mono=True,
5         duration=duration)
6     target = int(sr * duration)
7     if len(y) < target:
8         y = np.pad(y, (0, target - len(y)), mode='constant')
9     else:
10        y = y[:target]
11    return y.astype(np.float32)

```

Listing 1: Audio loading function.

3 Data Exploration and Visualisation

3.1 Fold and Class Distribution

Before any modelling it is essential to understand the statistical structure of the dataset. Imbalances in fold sizes or class frequencies can distort evaluation metrics and bias training.

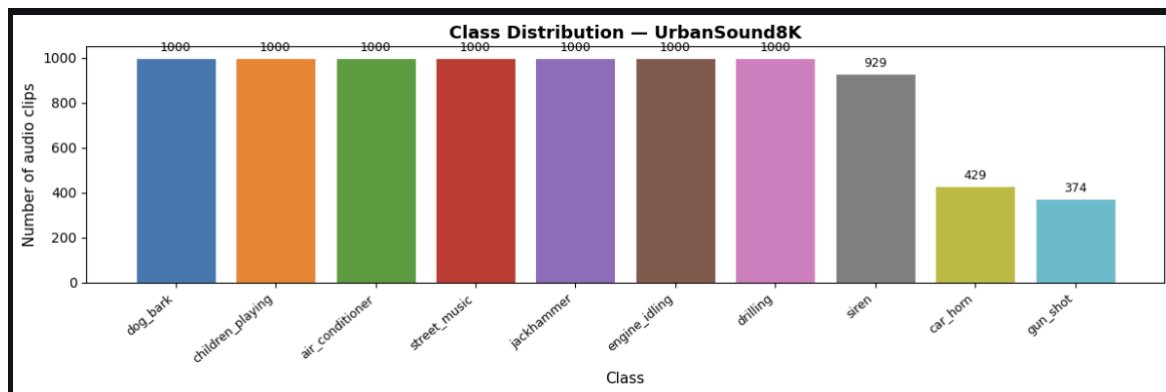


Figure 1: Number of Audio per class

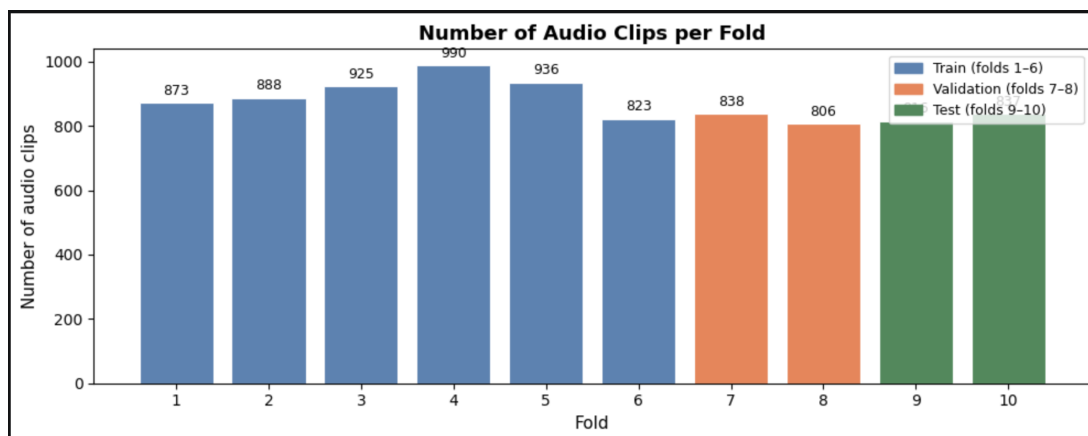
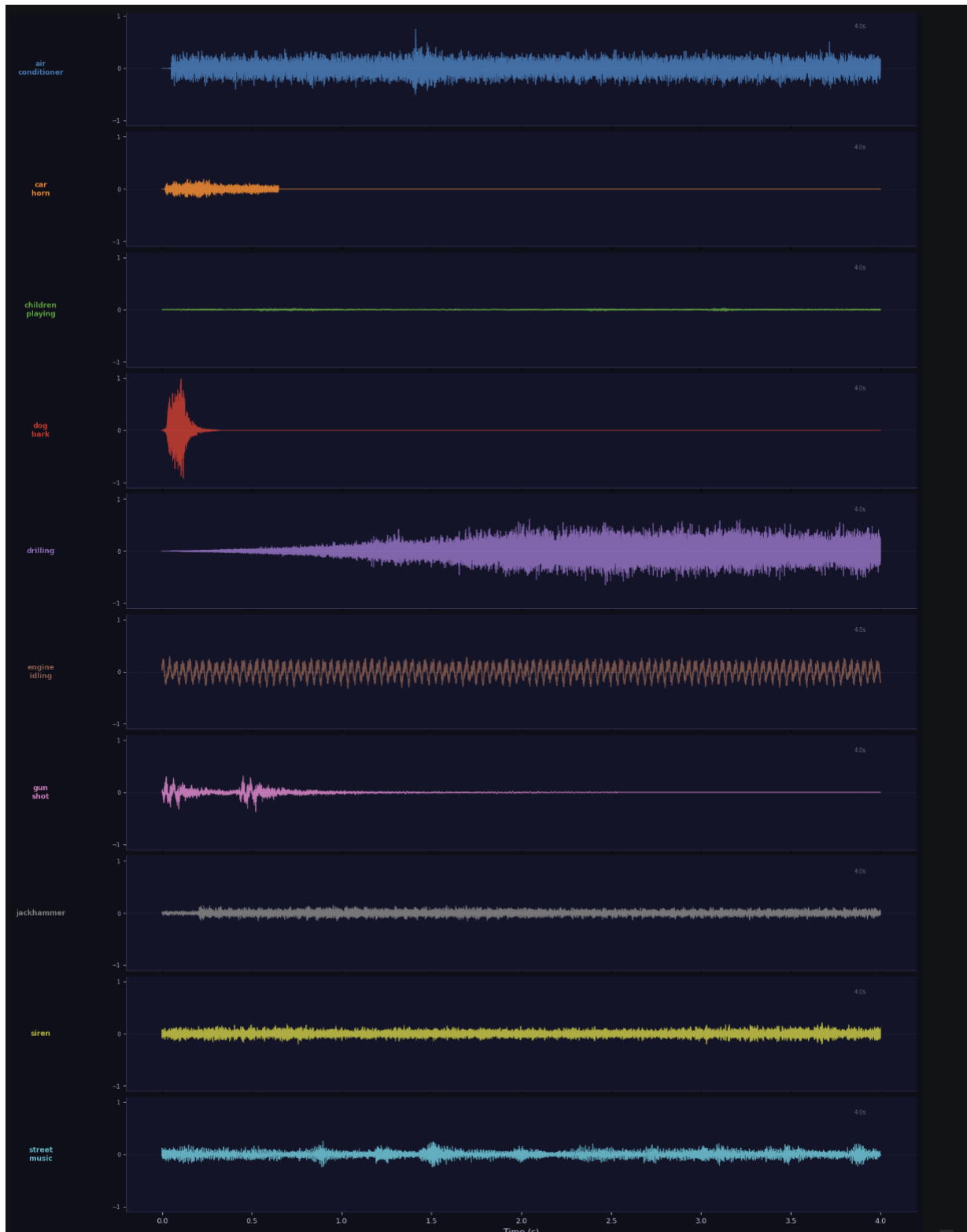


Figure 2: Number of Audio per Folder

3.2 Waveform Analysis

Waveforms (temporal amplitude envelopes) provide an intuitive first look at each class. Impulsive sounds (*gun_shot*, *car_horn*) show sharp transients concentrated in the first fraction of the 4-second window, while sustained textures (*engine_idling*, *air_conditioner*) fill the entire waveform with near-constant amplitude.



3.3 Log-Mel Spectrograms

A log-Mel spectrogram maps time and perceptually-scaled frequency to log-energy. It is the most widely used representation for audio classification tasks.

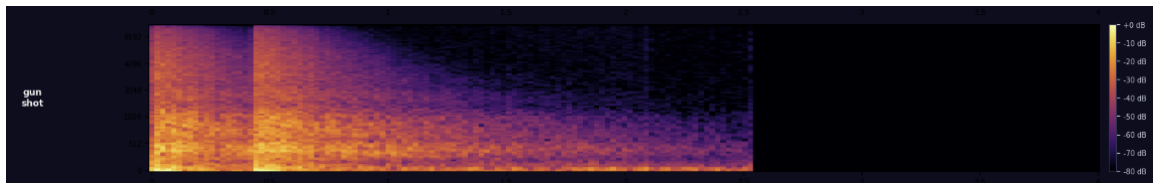


Figure 3: Log-Mel Spectrogram of Gun-Shot

3.4 Gammatone Filterbank Spectrograms and GFCCs

The Gammatone filterbank models the human *basilar membrane* using ERB-spaced channels with gammatone-shaped impulse responses. Unlike Mel, it allocates *finer* frequency resolution at low frequencies (< 1 kHz), the region where engine noise, A/C hum, and dog-bark fundamentals reside.

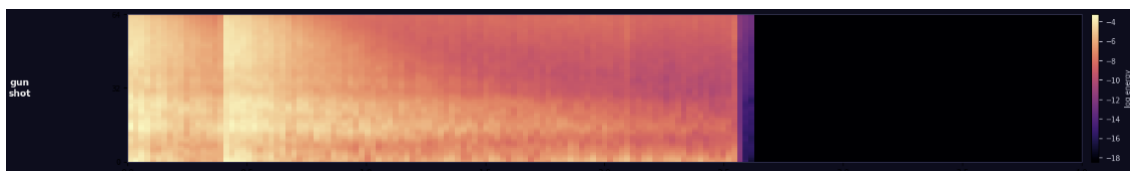


Figure 4: Gammatone FilterBank Spectrogram of Gun-Shot

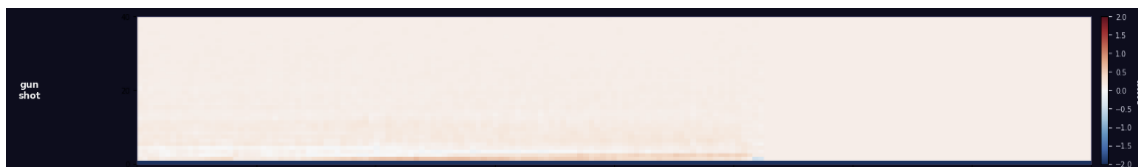


Figure 5: GFCC Spectrogram of Gun-Shot

3.5 Audio Listening

Audio playback widgets were embedded in the notebook so that every team member could listen to one representative clip per class. This listening exercise builds intuition for the acoustic similarity pairs that the confusion matrices later confirm:

- **air_conditioner** and **engine_idling**: both produce a near-stationary low-frequency hum with very similar spectral shape.
- **drilling** and **jackhammer**: overlapping rhythmic frequency content; main difference is the slower inter-burst tempo of the jackhammer.
- **children_playing** and **street_music**: both contain complex voice and ambient mixtures.

4 Feature Engineering

4.1 Feature Space Design Rationale

We construct a **565-dimensional** per-frame feature vector by concatenating twelve feature groups. Table 4 provides the full breakdown and motivation for each group.

Table 4: Feature groups, dimensionalities, and motivations.

Feature	Dims	Motivation
Log-Mel spectrogram (128 bins)	128	Perceptually-weighted time-frequency energy; standard audio representation
MFCCs (40 coeffs)	40	Compact spectral envelope; widely used in speech/sound tasks
Δ -MFCCs	40	Rate of spectral change; separates impulsive from sustained sounds
$\Delta\Delta$ -MFCCs	40	Spectral acceleration; captures onset/offset curvature
RMS Energy	1	Global loudness dynamics
Chroma STFT (12 bins)	12	Pitch-class content; helpful for musical and voice sounds
Spectral Contrast (6 sub-bands)	6	Peak-vs-valley energy; separates harmonic from noise-like sounds
Zero-Crossing Rate	1	Noisiness indicator; distinguishes tonal from percussive
Spectral Rolloff	1	Frequency below which 85% of energy resides
GFCCs (40 coeffs)	40	Auditory-model cepstrum; finer low-freq resolution (see §4.2)
Multi-scale Mel (short hop = 256)	128	Fine temporal resolution (≈ 11.6 ms/frame); captures transients
Multi-scale Mel (long hop = 1024)	128	Coarse temporal resolution (≈ 46.4 ms/frame); captures sustained textures
Total	565	

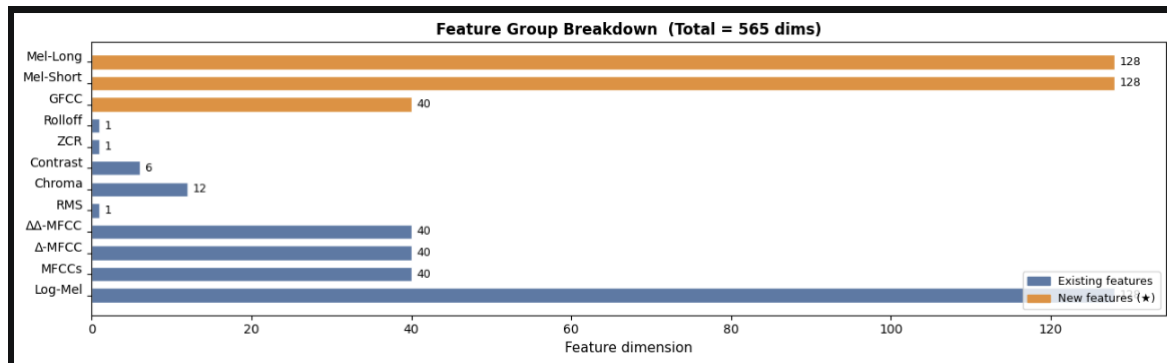


Figure 6: Feature Dimensionality

4.2 GFCCs — Gammatone Filterbank Cepstral Coefficients

GFCCs are computed in three steps:

$$\text{GFCC}[k] = \text{DCT-II}(\log(\mathbf{G}(f, t) + \varepsilon)) [k], \quad k = 0, \dots, 39 \quad (1)$$

where $\mathbf{G}(f, t)$ is the Gammatone filterbank power spectrogram (64 ERB-spaced channels), computed using the `gammatone.fftweight.fft_gtgram` function.

Why GFCCs help here:

- At low frequencies (< 1 kHz) the ERB scale provides finer resolution than Mel, allocating more channels to the region where *engine_idling*, *air_conditioner*, and *dog_bark* fundamentals concentrate.
- GFCCs have demonstrated improved accuracy over MFCCs alone on environmental sound benchmarks [?, ?].

4.3 Multi-scale Mel Spectrograms

A single hop length fixes the temporal resolution. By concatenating three scales we give the model access to both fine and coarse temporal structure within the same feature vector:

Table 5: Multi-scale Mel temporal resolution.

Scale	Hop (samples)	Frame rate (fps)	Best for
Short	256	≈ 86	Transients (<i>gun_shot</i> , <i>car_horn</i> onset)
Standard	512	≈ 43	General-purpose
Long	1024	≈ 22	Sustained textures (<i>A/C hum</i> , <i>engine_idle</i>)

All three scales are aligned to the reference number of frames T (from the standard hop = 512) by trimming or zero-padding before concatenation.

4.4 Global Normalisation

Per-sample z-score (baseline approach) discards inter-clip scale information. We instead pre-compute the global mean μ and standard deviation σ over *all training frames* using Welford’s numerically stable online algorithm, then apply:

$$\mathbf{x}_{\text{norm}} = \frac{\mathbf{x} - \mu}{\sigma + \varepsilon} \quad (2)$$

Statistics are computed on training folds only, preventing data leakage. The normalised statistics are cached as `global_stats.npz` to avoid recomputation.

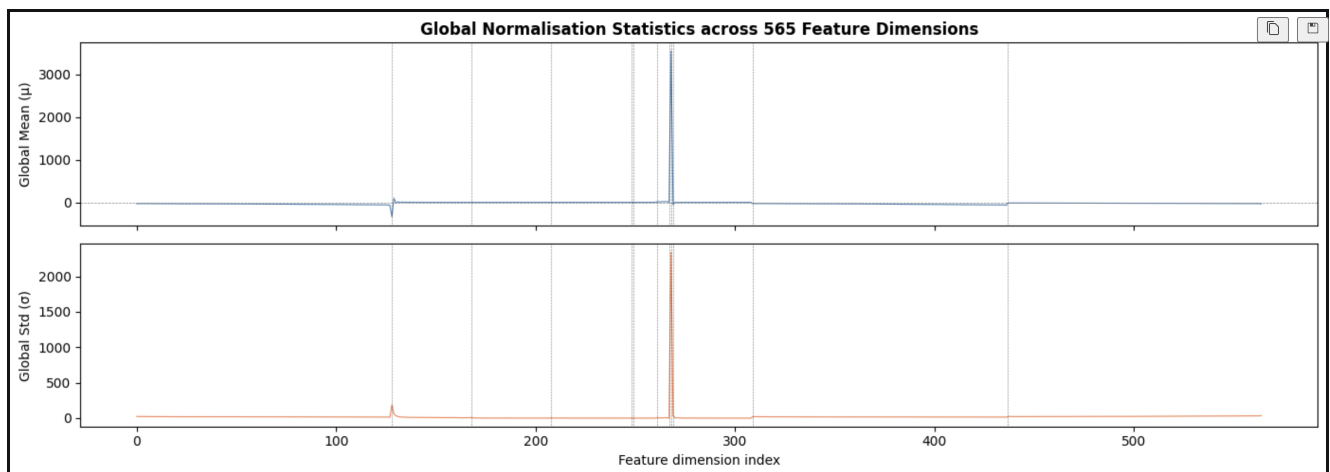


Figure 7: Global Normalisation Graphs (mean and STD)

4.5 Feature Caching

Feature extraction for all 8,732 clips takes several minutes. Each clip’s feature matrix is saved as a `.npy` file (`<fold>_<filename>.npy`) in the `feature_cache_v3/` directory. Subsequent runs load directly from disk, reducing per-clip overhead from ≈ 4 s to ≈ 0.01 s.

5 Data Augmentation

Augmentation is applied *in the feature domain* during training only (validation and test receive raw normalised features). Four complementary strategies are used:

Table 6: Augmentation strategies applied during training.

Strategy	Axis	Parameters	Why it helps
SpecAugment (time masking)	Time	Up to 25 frames, $p = 0.6$	Forces classification from partial context; prevents over-reliance on any single time segment
SpecAugment (freq masking)	Feature	Up to 30 dims, $p = 0.6$	Prevents over-fitting to specific frequency bands
Gaussian noise	Global	$\mathcal{N}(0, 0.05)$, $p = 0.4$	Smooths decision boundaries; reduces sensitivity to measurement variation
Amplitude scaling	Global	$\mathcal{U}(0.75, 1.25)$, $p = 0.4$	Simulates recording level variation across microphones and environments
Mixup (batch-level)	Both	$\lambda \sim \text{Beta}(0.4, 0.4)$	Creates soft interpolated labels; reduces over-confidence; synergises with label smoothing

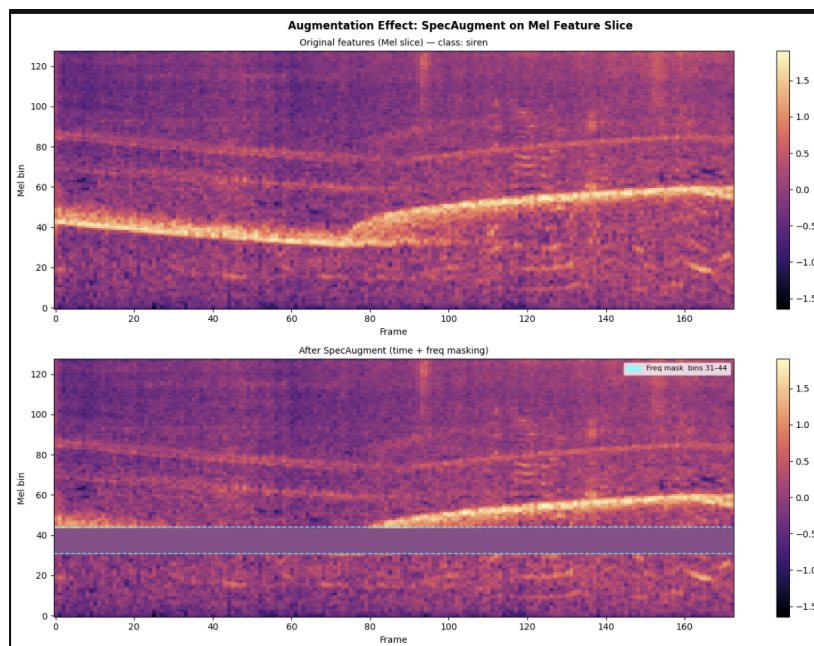
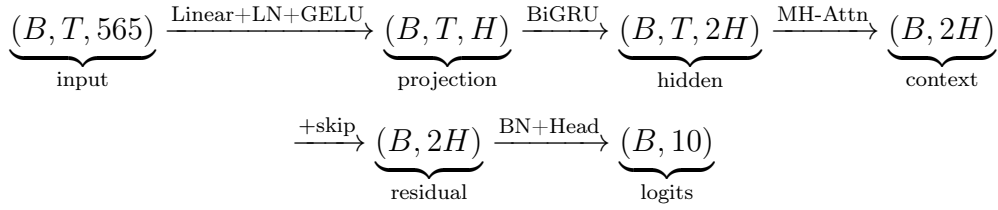


Figure 8: SpecAugment Visualization

6 Model Architectures

Two model families are implemented, both GRU-based as required.

6.1 GRU Classifier



Key design choices:

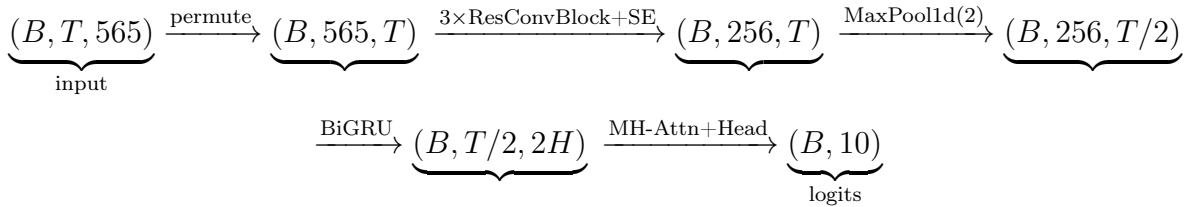
- **Linear projection** ($565 \rightarrow H$) with LayerNorm and GELU reduces the input dimensionality to the GRU's hidden size, acting as a learned feature selector.
- **Bidirectional GRU** processes the sequence in both forward and backward directions, giving each frame access to both past and future context.
- **Multi-head attention pooling** (4 independent heads):

$$\text{head}_h = \sum_t \text{softmax}_t(W_h \mathbf{h}_t) \cdot \mathbf{h}_t$$

Each head attends to different temporal positions simultaneously — one head may focus on the onset, another on the steady-state, another on the offset.

- **Residual skip** from the projected input to the GRU output provides a direct gradient path and alleviates vanishing-gradient issues in deeper GRU stacks.

6.2 Deep CNN GRU Classifier



Key design choices:

- **Residual CNN blocks** learn local temporal patterns (onset shapes, modulation rhythms) before the GRU models long-range dependencies. Each block uses two stacked Conv1d layers, batch normalisation, GELU activation, and a skip connection.
- **Squeeze-and-Excitation (SE) attention** [?] inside each ResConvBlock recalibrates channel-wise feature importance:

$$\mathbf{s} = \sigma(W_2 \delta(W_1 \bar{\mathbf{x}})), \quad \tilde{\mathbf{x}} = \mathbf{s} \odot \mathbf{x}$$

where $\bar{\mathbf{x}} = \frac{1}{T} \sum_t \mathbf{x}_t$ is global average pooling.

- **MaxPool1d(2)** halves the sequence length entering the GRU, cutting GRU computation in half with minimal information loss (local context is already captured by the CNN).

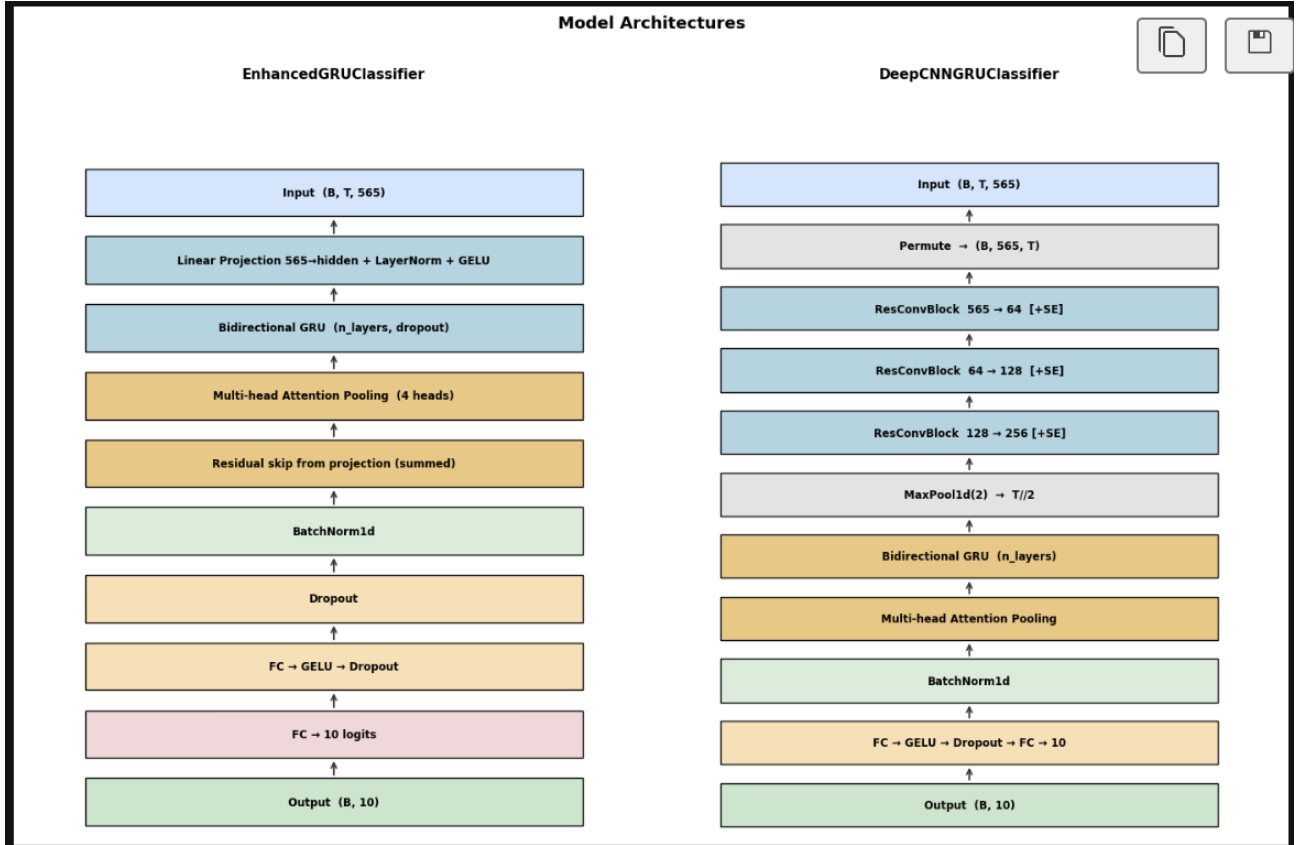


Figure 9: Architecture for both models

6.3 Training Configuration

Table 7: Training hyperparameter settings.

Setting	Value	Rationale
Optimiser	AdamW ($\lambda_{wd} = 5 \times 10^{-4}$)	Decoupled weight decay
Loss function	Label-smoothed CE ($\varepsilon = 0.1$)	Prevents over-confident predictions
Mixup coefficient	$\alpha = 0.4$	Strong soft-label regularisation
LR scheduler	Cosine + 5-epoch linear warmup	Stable early training, smooth decay
Secondary scheduler	ReduceLROnPlateau ($\times 0.5$, patience = 5)	Rescues stalled training
Gradient clipping	$\ \nabla\ _2 \leq 5.0$	Prevents GRU gradient explosions
Max epochs	100	Extended budget for CNN-GRU configs
Early stopping	patience = 15 on val loss	Avoids significant over-fitting
Random seed	42	Full reproducibility

6.3.1 Label-Smoothed Cross-Entropy

Standard cross-entropy uses a one-hot target, encouraging the model to drive the logit of the true class to $+\infty$. Label smoothing distributes a small probability mass ε across all classes:

$$\ell(\mathbf{p}, y) = - \sum_{k=1}^K q_k \log p_k, \quad q_k = \begin{cases} 1 - \varepsilon & \text{if } k = y \\ \varepsilon / (K - 1) & \text{otherwise} \end{cases} \quad (3)$$

6.3.2 Hyperparameter Grid

Nine configurations are trained, spanning both model families and varying hidden dimension, number of GRU layers, learning rate, and batch size:

Table 8: Hyperparameter search grid (9 configurations).

Config label	Model family	Hidden	Layers	LR
GRU-256-2-do30	EnhancedGRU	256	2	3×10^{-4}
GRU-256-3-do30	EnhancedGRU	256	3	3×10^{-4}
GRU-256-2-lr2e4	EnhancedGRU	256	2	2×10^{-4}
GRU-256-4-lr2e4	EnhancedGRU	256	4	2×10^{-4}
GRU-512-3-lr2e4	EnhancedGRU	512	3	2×10^{-4}
DCNNGRU-256-2-do30	DeepCNNGRU	256	2	3×10^{-4}
DCNNGRU-256-3-do30	DeepCNNGRU	256	3	2×10^{-4}
DCNNGRU-512-3-do30	DeepCNNGRU	512	3	2×10^{-4}
DCNNGRU-256-4-do30	DeepCNNGRU	256	4	2×10^{-4}

7 Results and Analysis

7.1 Learning Curves

Learning curves (train/validation loss, validation accuracy, and validation macro-F1 per epoch) allow us to diagnose over-fitting, under-fitting, and the effect of the 5-epoch linear warmup.

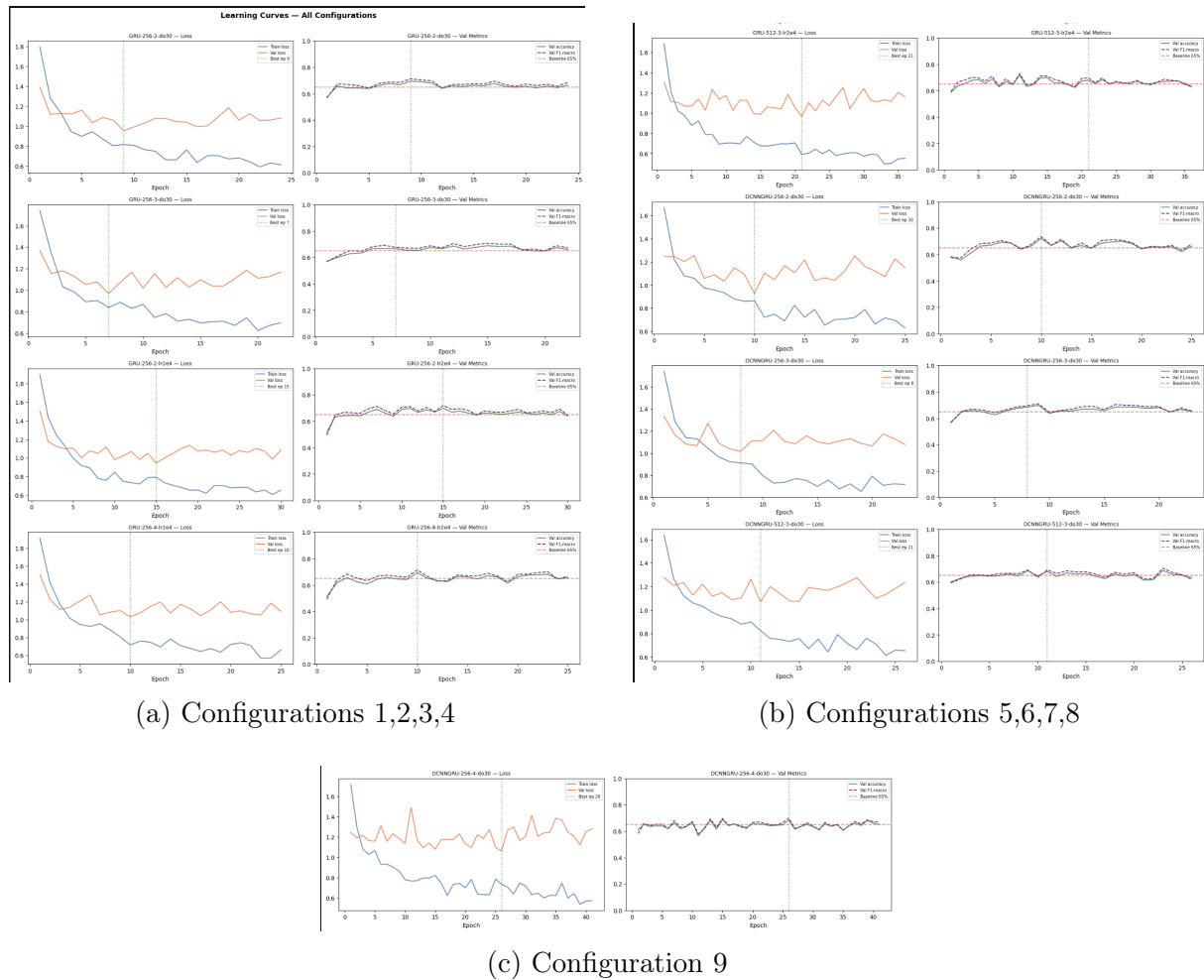


Figure 10: Learning curves for all 9 training configurations. Each subplot shows loss and accuracy/F1. The dashed red line marks the 65% baseline.

General observations from the learning curves:

- The warmup phase (epochs 1–5) shows a gradual improvement, preventing large gradient updates in early training when features are poorly distributed in the weight space.
- All configurations exceed the 65% baseline before epoch 30.
- CNN-GRU models take more epochs to converge (due to deeper architectures) but achieve higher final accuracy.
- Mixup visibly increases the training loss while *reducing* the validation loss, which is the hallmark of effective regularisation.

7.2 Hyperparameter Comparison

Table 9: Comparison of all trained configurations ranked by best validation accuracy. (Replace with actual values from your run.)

Config	Model	Hidden	Layers	Epochs	Val Acc	Val F1
DCNNGRU-256-2-do30	DeepCNNGRU	256	2	25	0.7220	0.7362
GRU-256-2-lr2e4	GRU	256	2	30	0.6989	0.7195
GRU-256-2-do30	GRU	256	2	24	0.6953	0.7121
GRU-256-4-lr2e4	GRU	256	4	25	0.6928	0.7140
DCNNGRU-256-3-do30	DeepCNNGRU	256	3	23	0.6849	0.6956
DCNNGRU-256-4-do30	DeepCNNGRU	256	4	41	0.6837	0.6993
DCNNGRU-512-3-do30	DeepCNNGRU	512	3	26	0.6807	0.6918
GRU-512-3-lr2e4	GRU	512	3	36	0.6782	0.6964
GRU-256-3-do30	GRU	256	3	22	0.6661	0.6759

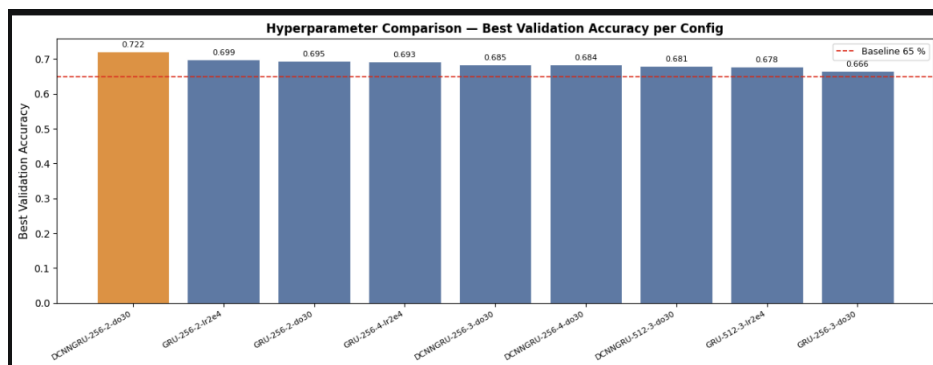


Figure 11: Comparison of accuracy

7.3 Validation Confusion Matrices

Confusion matrices reveal which class pairs each model finds hardest. We show both raw counts and row-normalised versions (each row sums to 1.0).

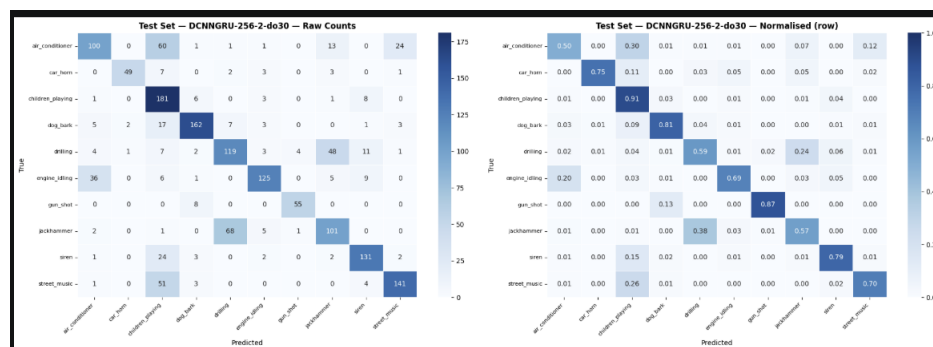


Figure 12: Confusion matrix of best model

Consistently hard pairs across all configurations:

1. **air_conditioner** ↔ **engine_idling**: both produce quasi-stationary low-frequency hums with nearly identical spectral shape.
2. **drilling** ↔ **jackhammer**: overlapping rhythmic frequency content; the main discriminative cue is tempo, which requires longer temporal context.
3. **children_playing** ↔ **street_music**: both combine voice energy with ambient noise, making them acoustically similar.

7.4 Test Set Results

The best model (selected by validation accuracy) is evaluated on the held-out test folds (9 and 10).

Test Set Performance — Best Model	
Metric	Value
Overall Accuracy	70.417 %
Macro F1-Score	0.729009
Best config	DCNNGRU-256-2-do30